

SecureBank

Rapport de Tests et Modifications

DIC2-SSI – ESP Dakar – Juin 2026

1. Contexte général

Ce document retrace l'ensemble des tests effectués et des modifications apportées au projet SecureBank dans le cadre du cours de Programmation Sécurisée DevSecOps (DIC2-SSI). Les opérations ont été réalisées sur un environnement local Windows 11 avec Java 25, Spring Boot 3.2, MySQL 8 via XAMPP et un frontend React/Vite.

Le backend se trouve dans le répertoire :

`C:\Users\HP\Downloads\secureBank\securebank_Backend\`

Le frontend se trouve dans le répertoire :

`C:\Users\HP\Downloads\secureBank\securebank_frontend\`

2. Environnement de développement

2.1 Stack technique

- Backend : Spring Boot 3.2 / Java 25 (compilé en cible Java 21)
- Base de données : MySQL 8 via XAMPP sur le port 3306
- Frontend : React 18 / Vite sur le port 3000
- IDE : VS Code avec terminal PowerShell/CMD
- Outil de build : Apache Maven 3.9.6 installé manuellement dans `C:\apache-maven-3.9.6\`
- Accès BDD : phpMyAdmin sur `http://localhost/phpmyadmin`

2.2 Commandes de démarrage

Démarrer le backend :

`cd securebank_Backend && mvn spring-boot:run -DskipTests`

Démarrer le frontend :

`cd securebank_frontend && npm run dev`

Libérer le port 8080 entre deux lancements :

`for /f "tokens=5" %a in ('netstat -ano ^| findstr :8080') do taskkill /PID %a /F`

3. Configuration effectuée

3.1 Base de données

La base de données `securebank_db` a été créée sur le serveur MySQL local via phpMyAdmin. Les tables ont été générées automatiquement par Hibernate au premier démarrage grâce à la propriété `ddl-auto: update`. Les tables créées sont : `users`, `accounts`, `transactions`, `audit_logs`, `otp_tokens`.

3.2 Fichier `application.yml`

Les paramètres suivants ont été configurés manuellement :

- URL datasource : `jdbc:mysql://localhost:3306/securebank_db` avec `characterEncoding=UTF-8`
- Identifiants MySQL : username `root`, password `securebank123`
- Email SMTP : `cheikhahmedtidianediengl@esp.sn` via Gmail App Password
- Clé JWT secrète de 256 bits minimum
- Encodage serveur forcé en UTF-8 pour éviter les erreurs de parsing JSON
- Swagger UI activé sur `/swagger-ui/index.html`

3.3 Mot de passe MySQL

Le mot de passe `root` MySQL a été défini via phpMyAdmin avec la commande SQL :

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'securebank123';  
FLUSH PRIVILEGES;
```

4. Problèmes rencontrés et solutions

4.1 Maven non reconnu (`CommandNotFoundException`)

Problème : La commande `mvn` n'était pas reconnue dans PowerShell car Maven n'était pas installé.

Solution : Téléchargement manuel de Apache Maven 3.9.6 depuis maven.apache.org et ajout au PATH système via la commande PowerShell :

```
[System.Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\apache-maven-3.9.6\bin",  
[System.EnvironmentVariableTarget]::User)
```

4.2 Lombok incompatible avec Java 25

Problème : 47 erreurs de compilation du type cannot find symbol (getEmail, getBalance, builder, etc.) car Lombok ne génère pas les méthodes sur Java 25.

Solution : Suppression complète de Lombok. Réécriture manuelle de tous les getters, setters et builders dans les 5 entités JPA (User, Account, Transaction, AuditLog, OtpToken) ainsi que dans tous les services et controllers.

4.3 Erreur Access denied for user root

Problème : Spring Boot ne pouvait pas se connecter à MySQL car le mot de passe dans application.yml ne correspondait pas.

Solution : Définition du mot de passe root MySQL via phpMyAdmin puis mise à jour du fichier application.yml en conséquence.

4.4 Port 8080 déjà utilisé

Problème : L'application ne démarrait pas car une instance précédente du backend occupait encore le port 8080.

Solution : Identification et kill du processus via netstat et taskkill. Ce problème s'est répété plusieurs fois lors des redémarrages.

4.5 Erreur 403 sur Swagger UI

Problème : L'accès à <http://localhost:8080/swagger-ui.html> retournait une erreur HTTP 403 car Spring Security bloquait l'URL avant la redirection.

Solution : Ajout des URLs Swagger dans la liste des endpoints publics de SecurityConfig.java :

```
/swagger-ui/**, /swagger-ui.html, /v3/api-docs/**, /webjars/**
```

L'URL correcte est <http://localhost:8080/swagger-ui/index.html>.

4.6 Erreur JSON circulaire (position 258321)

Problème : Lors de l'appel GET /api/accounts, le frontend recevait une réponse JSON de plus de 258 000 caractères causant une erreur de parsing. La cause était une boucle de sérialisation circulaire : User référence Account qui référence User qui référence Account indéfiniment.

Solution : Ajout de l'annotation @JsonIgnore sur toutes les relations bidirectionnelles dans les entités JPA :

- Dans User.java : @JsonIgnore sur la liste accounts et le champ password
- Dans Account.java : @JsonIgnore sur owner, sentTransactions et receivedTransactions
- Dans Transaction.java : @JsonIgnore sur sourceAccount et destinationAccount
- Ajout de @JsonIgnoreProperties({"hibernateLazyInitializer", "handler"}) sur Account et Transaction

4.7 Communications link failure (MySQL non démarré)

Problème : Le backend refusait de démarrer avec l'erreur Communications link failure / Connection refused car MySQL XAMPP n'était pas actif.

Solution : Démarrage de MySQL depuis le XAMPP Control Panel avant chaque session de développement.

4.8 Mockito incompatible avec Java 25

Problème : Les tests JUnit échouaient avec l'erreur Java 25 (69) is not supported by the current version of Byte Buddy car Mockito utilise Byte Buddy qui ne supporte officiellement que jusqu'à Java 22.

Solution : Utilisation du flag -DskipTests pour ignorer les tests au démarrage. Le test AuthServiceTest a été réécrit sans mocker JwtUtil pour contourner la limitation.

5. Modifications du code

5.1 Backend – Fichiers modifiés

SecurityConfig.java

Localisation : src/main/java/com/securebank/config/SecurityConfig.java

Modifications :

- Ajout des URLs Swagger dans la liste des endpoints publics
- Ajout de /v3/api-docs/**, /webjars/**, /swagger-ui.html dans requestMatchers

User.java / Account.java / Transaction.java

Localisation : src/main/java/com/securebank/model/

Modifications :

- Suppression de toutes les annotations Lombok (@Data, @Builder, @NoArgsConstructor, @AllArgsConstructor)
- Réécriture manuelle des getters, setters et builders pour chaque entité
- Ajout de @JsonIgnore sur toutes les relations bidirectionnelles pour casser les boucles de sérialisation
- Ajout de @JsonIgnoreProperties sur Account et Transaction

application.yml

Localisation : src/main/resources/application.yml

Modifications :

- Ajout de server.servlet.encoding.charset: UTF-8 et force: true

- Ajout de `characterEncoding=UTF-8` dans l'URL datasource
- Configuration des identifiants MySQL locaux
- Configuration de l'App Password Gmail pour l'envoi des OTP

pom.xml

Localisation : racine du projet

Modifications :

- Ajout de `project.build.sourceEncoding UTF-8`
- Ajout de `project.reporting.outputEncoding UTF-8`
- Suppression de la dépendance Lombok
- Ajout du plugin `maven-compiler-plugin` avec `encoding UTF-8`

5.2 Frontend – Fichiers modifiés

OtpPage.jsx

Localisation : `src/pages/OtpPage.jsx`

Modifications :

- Suppression du double appel `verifyOtp` + `confirmTransfer` lors d'un virement
- Pour le flux LOGIN : appel unique à `verifyOtp` qui retourne le JWT
- Pour le flux TRANSFER : appel unique à `confirmTransfer` qui vérifie l'OTP côté backend
- Décodage du JWT via `atob()` pour extraire le rôle et l'email sans librairie externe

TransferPage.jsx

Localisation : `src/pages/TransferPage.jsx`

Modifications :

- Suppression du appel `verifyOtp` inutile dans la fonction `confirm()`
- Un seul appel à `confirmTransfer` avec la référence UUID et le code OTP
- Ajout d'une fonction `reset()` pour réinitialiser proprement le formulaire après un virement réussi
- Amélioration de l'interface : indicateur d'étapes, récapitulatif du virement, écran de succès

App.jsx

Localisation : `src/App.jsx`

Modifications :

- Ajout de la vérification d'expiration du JWT au démarrage de l'application
- Si le token JWT stocké dans `localStorage` est expiré, il est automatiquement supprimé
- L'utilisateur est redirigé vers la page de login sans action manuelle

6. Tests effectués

6.1 Tests via curl (terminal)

Tous les tests ci-dessous ont été effectués depuis un second terminal pendant que le backend tournait.

Inscription d'un utilisateur

```
curl -X POST http://localhost:8080/api/auth/register -H "Content-Type: application/json"
-d '{"email":"test@test.com","password":"Test1234!","fullName":"Test
User","phone":"0700000000"}'
```

Résultat : {"success":true,"message":"Compte créé, en attente d'activation"}

Test de connexion sans activation

Résultat attendu et obtenu : message Compte non activé par l'administrateur.

Activation via phpMyAdmin

```
UPDATE users SET enabled = 1 WHERE email = 'test@test.com';
```

Résultat : 1 row affected.

Connexion après activation

Résultat : otpRequired: true + réception du code OTP par email dans les secondes qui suivent.

Validation OTP

Saisie du code à 6 chiffres reçu par email. Résultat : token JWT retourné, redirection vers le dashboard.

6.2 Tests via l'interface frontend

Création de compte bancaire

Type : Compte courant. Dépôt initial : 100 000 000 FCFA. Résultat : numéro de compte SBxxxxxxxxx généré et affiché.

Consultation du solde

Résultat : solde affiché correctement après correction de l'erreur de sérialisation JSON circulaire.

6.3 Gestion des utilisateurs en BDD

Les opérations suivantes ont été effectuées directement via phpMyAdmin :

- Création et suppression de comptes de test (test@test.com)
- Promotion d'un compte en ROLE_ADMIN via UPDATE users SET role = 'ROLE_ADMIN'
- Changement de mot de passe admin via injection directe d'un hash BCrypt généré
- Activation de comptes clients via UPDATE users SET enabled = 1

7. État final du projet

Au terme de cette session de développement et de tests, le projet SecureBank est dans l'état suivant :

- Backend Spring Boot opérationnel sur <http://localhost:8080>
- Frontend React opérationnel sur <http://localhost:3000>
- Base de données MySQL connectée avec 4 utilisateurs en BDD
- Swagger UI accessible sur <http://localhost:8080/swagger-ui/index.html>
- Flux d'authentification 2FA (mot de passe + OTP email) fonctionnel
- Création de comptes bancaires fonctionnelle
- Consultation du solde fonctionnelle
- Flux de virement en 2 étapes (initiation + confirmation OTP) implémenté
- Journal d'audit avec chaînage HMAC-SHA256 actif
- Chiffrement AES-256-GCM des données sensibles en place
- RBAC Spring Security actif : séparation ROLE_CLIENT / ROLE_ADMIN

Points restants à implémenter lors de la prochaine session :

Fonctionnalité	Où	Priorité
DE → VERS + Motif dans historique	Frontend + Backend DTO	Immédiat
Mot de passe oublié	Backend + Frontend	Haute
Verrouillage après 3 tentatives	Backend AuthService	Haute
Détection IP suspecte	Backend + Email	Moyenne
Expiration session auto	Frontend App.jsx	Moyenne
Statistiques admin	Backend + Frontend	Moyenne

Fonctionnalité	Où	Priorité
Filtres transactions admin	Backend + Frontend	Moyenne
Email récapitulatif après virement	Backend TransactionService	Basse
Export CSV transactions	Backend + Frontend	Basse